

Projeto ToDo List:

salvando no banco de dados

Até aqui, as tarefas sumiam ao recarregar a página. Nesta aula, o agente leva o ToDo List para um banco **SQLite**: as tarefas ficam guardadas num arquivo e sobrevivem a recarregar a página e a reiniciar o app. Você vê a tabela, as consultas parametrizadas, a separação em `db.py` e como conferir o banco.

SQLite

sqlite3 (biblioteca padrão)

db.py

CREATE · INSERT · UPDATE · DELETE

Consultas parametrizadas

.gitignore do banco

Formação Claude Code 2026 · Baseado em docs.python.org (sqlite3), sqlite.org e docs.streamlit.io, além de cursor.com/docs (Agent, Plan Mode, MCP), consultados em outubro de 2026 · Código de referência testado em `projeto/` · Telas: print real

Por que *um banco?*

AÇÃO	V2 (SESSION_STATE)	V3 (SQLITE)
Recarregar a página (F5)	Perde tudo	Mantém
Abrir em outra aba	Lista vazia	Mesma lista
Reiniciar o app	Perde tudo	Mantém
Fazer backup	Não dá	Copiar o arquivo tarefas.db

POR QUE SQLITE

O SQLite é um banco de dados **num único arquivo**, sem servidor para instalar. O Python já traz o módulo `sqlite3` na biblioteca padrão: nenhuma dependência nova, o que combina com a rule do projeto ("sem bibliotecas novas sem perguntar").

Para muitos usuários ao mesmo tempo, num servidor, o caminho seria PostgreSQL ou MySQL. Para um app pessoal ou de estudo, o SQLite resolve.

Planejar *antes de migrar*

Mudar onde os dados moram mexe em várias partes do app. É um bom caso para o modo **Plan**:

Planeje a migração do ToDo List para SQLite:

- as tarefas passam a ficar num arquivo `tarefas.db`, ao lado do `app.py`;
- crie um módulo `db.py` com as funções de acesso (`criar tabela`, `listar`, `adicionar`, `marcar`, `remover`, `remover concluídas`);
- o `app.py` só chama essas funções;
- use o `sqlite3` da biblioteca padrão e consultas parametrizadas;
- tudo o que funciona hoje continua igual.

Mostre o esquema da tabela. Não escreva código.

O QUE CONFERIR NO PLANO

- ✓ Uma tabela `tarefas` com `id`, `texto`, `concluída`
- ✓ O `id` vem do banco (`AUTOINCREMENT`), não mais do contador da sessão
- ✓ Funções pequenas, uma por operação
- ✓ Consultas com `?`, nunca montando SQL com f-string
- ✓ `tarefas.db` no `.gitignore`

A tabela *tarefas*

```
"""Acesso ao banco SQLite das tarefas."""
import sqlite3
from pathlib import Path

CAMINHO_BANCO = Path(__file__).with_name("tarefas.db")

def conectar(caminho=CAMINHO_BANCO):
    conexao = sqlite3.connect(caminho)
    conexao.row_factory = sqlite3.Row
    return conexao

def criar_tabela(conexao):
    conexao.execute(
        """
        CREATE TABLE IF NOT EXISTS tarefas (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            texto TEXT NOT NULL,
            concluida INTEGER NOT NULL DEFAULT 0,
            criada_em TEXT NOT NULL DEFAULT (datetime('now', 'localtime'))
        )
        """
    )
    conexao.commit()
```

COLUNA	TIPO	PARA QUÊ
id	INTEGER PK	Gerado pelo banco; nunca se repete
texto	TEXT NOT NULL	O texto da tarefa
concluida	INTEGER 0/1	O SQLite não tem booleano; 0 = não, 1 = sim
criada_em	TEXT	Data e hora, preenchida sozinha
CREATE TABLE IF NOT EXISTS cria a tabela só na primeira vez. Rodar de novo não apaga nada.		

As operações *em db.py*

```
def listar(conexao):
    linhas = conexao.execute("SELECT id, texto, concluida FROM tarefas ORDER BY id").fetchall()
    return [{"id": l["id"], "texto": l["texto"], "concluida": bool(l["concluida"])} for l in linhas]

def adicionar(conexao, texto):
    conexao.execute("INSERT INTO tarefas (texto) VALUES (?)", (texto,))
    conexao.commit()

def marcar(conexao, id_tarefa, concluida):
    conexao.execute("UPDATE tarefas SET concluida = ? WHERE id = ?", (int(concluida), id_tarefa))
    conexao.commit()

def remover(conexao, id_tarefa):
    conexao.execute("DELETE FROM tarefas WHERE id = ?", (id_tarefa,))
    conexao.commit()

def remover_concluidas(conexao):
    conexao.execute("DELETE FROM tarefas WHERE concluida = 1")
    conexao.commit()
```

FUNÇÃO	SQL
listar	SELECT
adicionar	INSERT
marcar	UPDATE
remover · remover_concluidas	DELETE

Cada escrita termina com `commit()` : sem ele, a mudança não fica gravada no arquivo.

`listar()` devolve a mesma lista de dicionários da v2. Por isso o resto do `app.py` quase não muda.

Consultas *parametrizadas*

CERTO: O VALOR VAI SEPARADO, NO LUGAR DO ?

```
conexao.execute("INSERT INTO tarefas (texto) VALUES (?)", (texto,))
```

ERRADO: O TEXTO DO USUÁRIO VIRA PARTE DO SQL

```
# NÃO faça assim
conexao.execute(f"INSERT INTO tarefas (texto) VALUES ('{texto}')
```

Com a versão errada, uma tarefa como `x'); DROP TABLE tarefas;--` poderia mudar o comando. É a **injeção de SQL**. Uma simples aspa (*O'Reilly*) já quebraria o INSERT.

TESTADO NO CÓDIGO DE REFERÊNCIA

A tarefa `Ler "O'Reilly"; DROP TABLE tarefas;--` foi salva e lida **como texto**, com aspas e tudo, e a tabela continuou inteira.

Inclua na rule do projeto: *"SQL sempre com parâmetros ?, nunca com f-string"*. O agente segue a regra em todas as conversas.

O app.py *usando o banco*

```
# As tarefas agora ficam no arquivo tarefas.db: sobrevivem a recarregar a página
# e a reiniciar o app.
conexao = db.conectar()
db.criar_tabela(conexao)

with st.form("nova_tarefa", clear_on_submit=True):
    col_texto, col_botao = st.columns([4, 1], vertical_alignment="bottom")
    texto = col_texto.text_input("Nova tarefa", placeholder="O que você precisa fazer?")
    adicionar = col_botao.form_submit_button("Adicionar", width="stretch")

if adicionar:
    texto = texto.strip()
    if not texto:
        st.warning("Digite uma tarefa antes de adicionar.")
    else:
        db.adicionar(conexao, texto)

tarefas = db.listar(conexao)
```

```
if marcada != tarefa["concluida"]:
    db.marcar(conexao, tarefa["id"], marcada)
    st.rerun()

if col_remove.button("Remover", key=f"remove_{tarefa['id']}"):
    db.remover(conexao, tarefa["id"])
    st.rerun()

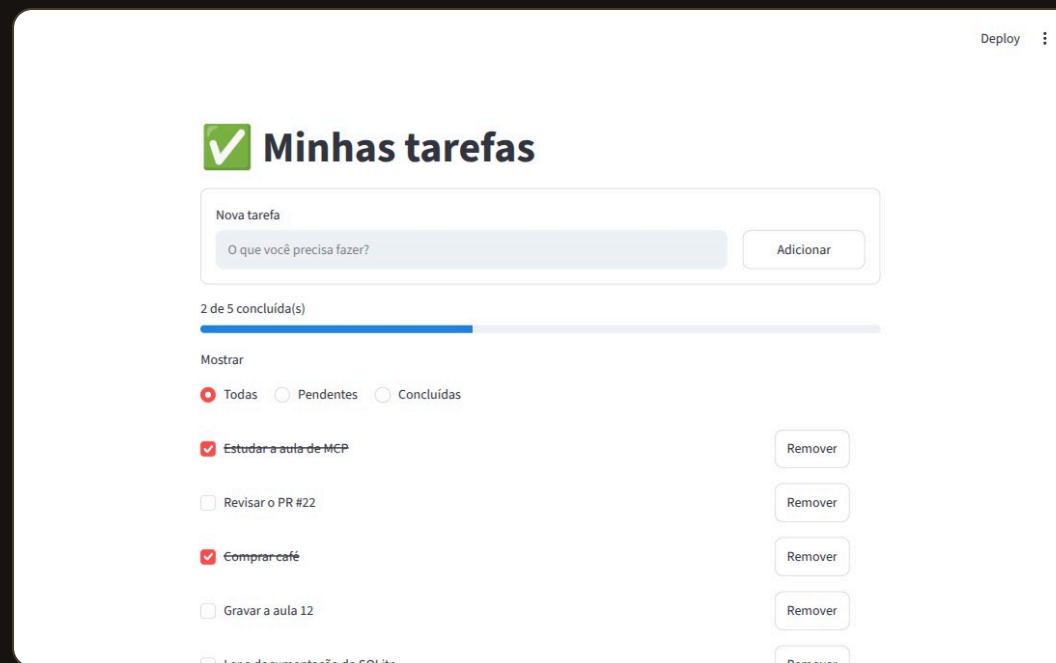
if concluidas and st.button("Limpar concluídas"):
    db.remover_concluidas(conexao)
    st.rerun()
```

- ✓ Sai o `st.session_state.tarefas` ; entra `db.listar()`
- ✓ Adicionar, marcar, remover e limpar viram chamadas ao `db`
- ✓ Progresso, filtro e checkboxes continuam iguais à v2
- ✓ As chaves continuam pelo `id`, agora o `id` do banco

Separar o acesso a dados em `db.py` deixa o `app.py` cuidando só da tela. Se um dia o banco mudar, só o `db.py` muda.

Versão simples: o app abre uma conexão a cada execução do script. Para apps maiores, peça ao agente para reutilizar a conexão com `st.cache_resource` ou fechá-la ao fim.

A versão 3 *funcionando*



Print real do `projeto/` da aula, lendo cinco tarefas de um `tarefas.db` de exemplo

O teste *que importa: persistência*

- 1 Adicione três tarefas e marque uma.
- 2 Recarregue a página (`F5`): tudo continua lá.
- 3 Pare o app (`Ctrl` + `C`) e rode de novo: tudo continua lá.
- 4 Abra o app em outra aba: a mesma lista.
- 5 Remova, limpe as concluídas e repita o F5.

@Browser Abra o app, adicione "Teste 1" e "Teste 2", marque "Teste 1" e recarregue a página. As duas tarefas e a marcação continuam? Depois remova "Teste 2" e recarregue de novo. Relate cada passo.

O teste automatizado da referência simula isso abrindo o app duas vezes com o `AppTest` : a segunda "sessão" encontra as tarefas e a marcação gravadas no banco.

Olhando *dentro do banco*

PELO PYTHON, SEM INSTALAR NADA

```
python -c "import sqlite3;
print(sqlite3.connect('tarefas.db').execute('SELECT * FROM
tarefas').fetchall())"
```

PELA FERRAMENTA SQLITE3 (SE INSTALADA)

```
sqlite3 tarefas.db "SELECT id, texto, concluida FROM tarefas;"
```

No Windows, a ferramenta `sqlite3` não vem instalada; o comando com Python funciona em todos os sistemas.

OU PEÇA AO AGENTE

No modo **Ask**: *"Mostre o conteúdo da tabela tarefas do tarefas.db"*. Ele roda a consulta no terminal, com sua aprovação, e explica o resultado.

OU COM MCP

Existem servidores MCP para bancos de dados (aula 07). Com um deles, o agente consulta o esquema e os dados direto, sem comandos de terminal. Para um arquivo SQLite local, o terminal já resolve.

O banco *fica fora do Git*

```
# .gitignore
.venv/
__pycache__/\ntarefas.db
```

- ✓ Código vai para o Git; dados não
- ✓ Cada pessoa que clonar começa com o banco vazio, criado na primeira execução
- ✓ Backup do banco é copiar o arquivo, não commitar

```
git add app.py db.py .gitignore
git commit -m "ToDo List: salva as tarefas em SQLite"
```

Se o `tarefas.db` já entrou num commit, só o `.gitignore` não basta:
rode `git rm --cached tarefas.db` e faça um commit.

O Todo List *em três aulas*

VERSÃO	O QUE ENTROU	RECURSOS DO CURSOR USADOS	LIÇÃO
v1 · aula 10	Adicionar, listar, remover	Rule, Plan, Agent, Browser	Escopo pequeno e critério de pronto
v2 · aula 11	Concluídas, progresso, filtro	Agent, Browser, Debug	Chave de widget pelo id; testar combinações
v3 · aula 12	Banco SQLite	Plan, Agent, Ask, terminal	Separar dados da tela; SQL parametrizado

Próximas ideias para treinar sozinho: **editar** o texto de uma tarefa, **prazo** com data, **categorias** e ordenação. Cada uma é um pedido pequeno, com teste e commit.

Teste o que *ficou*

1. Por que o SQLite combina com este projeto?

A) Precisa de um servidor dedicado

B) É um arquivo só e já vem no Python

C) Só roda na nuvem

D) É obrigatório no Streamlit

B. Nenhuma dependência nova.

2. Qual linha é segura?

A) `execute(f"... VALUES ('{texto}')`

B) `execute("... VALUES (?)", (texto,))`

C) As duas

D) Nenhuma

B. Parâmetros evitam injeção de SQL.

3. O que acontece se esquecer o `commit()` depois de um `INSERT`?

A) Nada muda

B) A tarefa pode não ficar gravada no arquivo

C) O banco é apagado

D) O app não abre

B. Toda escrita precisa de commit.

4. O `tarefas.db` deve ir para o Git?

A) Sim, sempre

B) Não: é dado, vai no `.gitignore`

C) Só no primeiro commit

D) Só se estiver vazio

B. Código no Git; dados fora.

Tarefas que *não somem mais*

O ToDo List agora guarda tudo num **SQLite**: uma tabela `tarefas`, um `db.py` com uma função por operação, consultas com `?` e `commit()` em cada escrita. O teste que importa é a **persistência**: recarregar, reiniciar e conferir. O banco fica fora do Git.

sqlite3

db.py

SQL parametrizado

commit()

F5 e reiniciar

.gitignore

Próxima aula → 13 · Pedra, Papel e Tesoura · Primeira Versão

Fontes: docs.python.org/3/library/sqlite3.html; sqlite.org (CREATE TABLE, datatypes); docs.streamlit.io (st.cache_resource, AppTest); cursor.com/docs (Plan Mode, Agent, MCP), consultados em outubro de 2026. Código de referência em `projeto/`, testado com Python 3.12 e Streamlit 1.65.